

KAIROS

Sistema de Análise Pedagógica com ICP

Relatório Técnico e Documentação

Versão 1.0
Janeiro de 2026

Conteúdo

1	Introdução	2
1.1	Objetivo do Sistema	2
1.2	Principais Funcionalidades	2
2	Estrutura do Sistema	2
2.1	Público-Alvo	2
2.2	Arquitetura do Sistema	3
3	Parâmetros e Métricas	3
3.1	Parâmetros TRI	3
3.2	Índices de Consistência	3
4	Implementação Técnica	3
4.1	Função: Cálculo de Parâmetros TRI	3
4.2	Função: Estimação de Proficiência	5
5	Fluxo de Dados	6
5.1	Processo de Análise	6
6	Resultados e Interpretação	6
6.1	Interpretação dos Resultados	7
7	Considerações Técnicas	7
7.1	Requisitos do Sistema	7
8	Conclusão	7
8.1	Próximos Desenvolvimentos	8

1 Introdução

1.1 Objetivo do Sistema

O KAIROS é um sistema de análise pedagógica que utiliza a Teoria de Resposta ao Item (TRI) e o Índice de Consistência Pedagógica (ICP) para fornecer insights sobre o desempenho dos alunos e a qualidade dos itens de avaliação.

1.2 Principais Funcionalidades

- Cálculo de parâmetros TRI (dificuldade e discriminação)
- Estimação de proficiência dos alunos
- Análise de consistência pedagógica
- Relatórios automáticos e visualizações
- Integração com dados educacionais

2 Estrutura do Sistema

2.1 Público-Alvo

kairos-blue!20 Público-Alvo	Necessidades
Professores	Análise de desempenho da turma, identificação de dificuldades
Coordenadores	Análise de desempenho da instituição, avaliação de metodologias
Pesquisadores	Dados para estudos psicométricos, validação de instrumentos
Desenvolvedores	Desenvolvimento de sistemas educacionais, APIs
Gestores Educacionais	Tomada de decisão baseada em dados, planejamento pedagógico

Tabela 1: Público-alvo e necessidades do sistema KAIROS

2.2 Arquitetura do Sistema

kairos-blue!20 Camada	Tecnologias	Função
Apresentação	Streamlit, Plotly, Matplotlib	Interface web e visualizações
Lógica de Negócio	Python, Pandas, NumPy	Processamento e análise de dados
Modelos	Scikit-learn, Custom TRI	Algoritmos de análise pedagógica
Dados	Pandas, CSV/Excel	Armazenamento e manipulação de dados
API	FastAPI (opcional)	Integração com outros sistemas

Tabela 2: Arquitetura técnica do sistema KAIROS

3 Parâmetros e Métricas

3.1 Parâmetros TRI

kairos-blue!20 Parâmetro	Descrição	Escala
Dificuldade (b)	Mede quão difícil é um item	-3 a 3
Discriminação (a)	Mede o poder discriminativo do item	0.3 a 2.5
Proficiência (θ)	Habilidade estimada do aluno	-3 a 3
Erro padrão (SE)	Precisão da estimativa de proficiência	0 a 1

Tabela 3: Parâmetros da Teoria de Resposta ao Item

3.2 Índices de Consistência

kairos-blue!20 Índice	Interpretação	Valores
ICP Verde	Respostas sólidas e previsíveis	0.8 - 1.0
ICP Amarelo	Oscilações que merecem atenção	0.5 - 0.79
ICP Vermelho	Possível chute sistemático	0.0 - 0.49

Tabela 4: Interpretação do Índice de Consistência Pedagógica

4 Implementação Técnica

4.1 Função: Cálculo de Parâmetros TRI

Listing 1: Cálculo de parâmetros TRI

```

1 def calcular_parametros_tri(respostas_binarias):
2     """
3     Calcula par metros TRI (dificuldade e discriminação) dos itens
4     .

```

```
5      Esta função implementa o modelo 2PL da Teoria de Resposta ao
6      Item
7      para estimar os parâmetros de dificuldade (b) e discriminação
8      (a).
9
10     Parâmetros:
11     -----
12     respostas_binarias : pandas.DataFrame
13         DataFrame com respostas binárias (0=erro, 1=acerto)
14
15     Retorna:
16     -----
17     dict : Dicionário com os seguintes parâmetros:
18         - 'dificuldade': array com parâmetro b de cada item
19         - 'discriminacao': array com parâmetro a de cada item
20     """
21     try:
22         # Obter dimensões da matriz de respostas
23         n_alunos, n_itens = respostas_binarias.shape
24
25         # Verificar se há dados suficientes
26         if n_alunos < 10:
27             st.warning("Número mínimo de alunos recomendado: 10")
28         if n_itens < 5:
29             st.warning("Número mínimo de itens recomendado: 5")
30
31         # 1. CÁLCULO DO PARÂMETRO DE DIFICULDADE (b)
32         proporcao_acertos = respostas_binarias.mean()
33         dificuldade = 1 - proporcao_acertos
34
35         # Normalizar dificuldade para escala padrão (-3 a 3)
36         dificuldade = 2 * (dificuldade - 0.5) * 3
37
38         # 2. CÁLCULO DO PARÂMETRO DE DISCRIMINAÇÃO (a)
39         discriminacao = np.zeros(n_itens)
40
41         for i, coluna in enumerate(respostas_binarias.columns):
42             try:
43                 # Calcular correlação item-total (ponto-bisserial)
44                 correlacao = respostas_binarias[coluna].corr(
45                     respostas_binarias.sum(axis=1) -
46                     respostas_binarias[coluna]
47                 )
48
49                 # Converter correlação para discriminação TRI
50                 if abs(correlacao) < 0.999:
51                     discriminacao[i] = correlacao / np.sqrt(1 -
52                         correlacao**2)
53                 else:
```

```
50         discriminacao[i] = 3.0
51
52         # Limitar valores extremos
53         discriminacao[i] = np.clip(discriminacao[i], 0.3,
54                                   3.0)
55
56     except Exception as e:
57         st.warning(f"Erro no cálculo de discriminação para
58                   {coluna}: {e}")
59         discriminacao[i] = 0.7
60
61     # 3. VALIDA O DOS PAR METROS
62     baixa_discriminacao = sum(discriminacao < 0.4)
63     if baixa_discriminacao > 0:
64         st.warning(f"          {baixa_discriminacao} itens com
65                   baixa discriminação")
66
67     return {
68         'dificuldade': dificuldade.values,
69         'discriminacao': discriminacao
70     }
71
72 except Exception as e:
73     st.error(f"Erro no cálculo dos par metros TRI: {e}")
74     return None
```

4.2 Função: Estimação de Proficiência

Listing 2: Estimação de proficiência TRI

```
1 def estimar_proficiencia_tri(respostas_binarias, parametros_tri):
2     """
3     Calcula proficiências dos alunos usando estimação TRI.
4
5     Esta função estima a proficiência (theta) de cada aluno
6     usando o método de máxima verossimilhança condicional.
7
8     Par metros:
9     -----
10    respostas_binarias : pandas.DataFrame
11        Respostas binárias dos alunos
12    parametros_tri : dict
13        Par metros dos itens calculados pela função anterior
14
15    Retorna:
16    -----
17    numpy.array : Array com proficiências estimadas
18    """
19    try:
20        # Extrair par metros dos itens
21        a = parametros_tri['discriminacao']
```

```

22     b = parametros_tri['dificuldade']
23
24     n_alunos = len(respostas_binarias)
25     proficiencias = np.zeros(n_alunos)
26
27     for i in range(n_alunos):
28         respostas = respostas_binarias.iloc[i].values
29
30         # Função de verossimilhança
31         def likelihood(theta):
32             prob = 1 / (1 + np.exp(-a * (theta - b)))
33             log_lik = np.sum(respostas * np.log(prob + 1e-10) +
34                             (1 - respostas) * np.log(1 - prob +
35                                                         1e-10))
36
37             return -log_lik
38
39         # Maximizar verossimilhança
40         result = minimize(likelihood, x0=0, bounds=[(-3, 3)])
41         proficiencias[i] = result.x[0]
42
43     return proficiencias
44
45 except Exception as e:
46     st.error(f"Erro na estimação de proficiência: {e}")
47     return None

```

5 Fluxo de Dados

5.1 Processo de Análise

kairos-blue!20 Etapa	Entrada	Saída
1. Coleta	Dados brutos (CSV/Excel)	Matriz de respostas
2. Limpeza	Matriz de respostas	Dados tratados
3. Análise TRI	Dados tratados	Parâmetros dos itens
4. Estimação	Parâmetros + Respostas	Proficiências
5. Cálculo ICP	Todos os dados anteriores	Índices de consistência
6. Relatório	Todas as análises	Relatório completo

Tabela 5: Fluxo de dados do sistema KAIROS

6 Resultados e Interpretação

6.1 Interpretação dos Resultados

kairos-blue!20 Métrica	Valor Ideal	Ação Recomendada
Discriminação (a)	0.8 - 1.5	Manter o item
Discriminação baixa	< 0.4	Revisar ou remover o item
Dificuldade (b)	-1 a 1	Adequado para maioria
Dificuldade extrema	< -2 ou > 2	Avaliar relevância
ICP Verde	> 0.8	Padrão esperado
ICP Vermelho	< 0.5	Investigar inconsistências

Tabela 6: Interpretação e ações baseadas nos resultados

7 Considerações Técnicas

7.1 Requisitos do Sistema

kairos-blue!20 Requisito	Especificação
Python	Versão 3.8 ou superior
Memória RAM	Mínimo 4GB (8GB recomendado)
Espaço em disco	500MB para dados e análises
Número de alunos	Mínimo 10 (recomendado > 30)
Número de itens	Mínimo 5 (recomendado > 10)
Formato de entrada	CSV, Excel (UTF-8 recomendado)

Tabela 7: Requisitos técnicos do sistema KAIROS

8 Conclusão

O sistema KAIROS fornece uma solução completa para análise pedagógica baseada em TRI e ICP. Suas principais vantagens incluem:

- **Escalabilidade:** Capaz de processar grandes volumes de dados
- **Precisão:** Utiliza métodos estatísticos robustos
- **Usabilidade:** Interface intuitiva baseada em Streamlit
- **Flexibilidade:** Adaptável a diferentes contextos educacionais
- **Interpretabilidade:** Resultados claros e acionáveis

8.1 Próximos Desenvolvimentos

Funcionalidade	Descrição	Prioridade
Integração com LMS	Conexão com Moodle, Google Classroom	Alta
Análise longitudinal	Acompanhamento ao longo do tempo	Média
Relatórios customizados	Templates personalizáveis	Baixa
API RESTful	Integração com outros sistemas	Alta
Dashboard em tempo real	Monitoramento contínuo	Média

Tabela 8: Roadmap de desenvolvimento futuro

Observações finais: Este sistema foi desenvolvido para apoiar educadores e gestores na tomada de decisão baseada em dados. Recomenda-se sempre considerar o contexto pedagógico específico ao interpretar os resultados.